UFPR / SOCM

Resolução de Questão de Prova: Inversão de Prioridade & Seção Crítica

Método Procedimental dos Marcos Críticos • Folha de Consulta

📋 Parâmetros da Questão (Moodle)

t1: Ingresso 0 | Dur 5 | Prio 2
• Usa PMC por 2 u.t.

t2: Ingresso 3 | Dur 6 | Prio 3
• Não acessa a PMC

t3: Ingresso 5 | Dur 3 | Prio 5
• Usa PMC por 2 u.t.

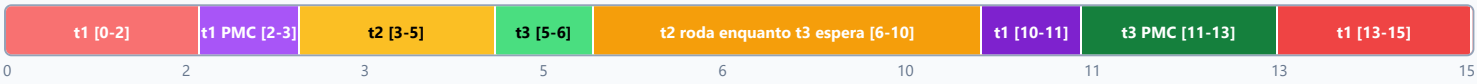
- Escalonador **preemptivo por prioridades** (maior valor = maior prioridade: 5 > 3 > 2).
- **Sem envelhecimento (aging)** e **SEM herança de prioridades** (fenômeno clássico).
- **PMC**: Página de Memória Compartilhada protegida por exclusão mútua.
- **t1** solicita e recebe acesso exclusivo à PMC no instante **t = 2**.
- **t3** solicita acesso exclusivo à PMC **1 unidade após o seu início**.

🎯 Gabarito Oficial dos Campos da Prova

Campo Solicitado	Instante
t1 solicita acesso exclusivo a PMC	2
t1 recebe acesso exclusivo a PMC	2
t1 libera a PMC	11
t3 solicita acesso exclusivo a PMC	6
t3 recebe acesso exclusivo a PMC	11
t3 libera a PMC	13
t2 termina sua execução	10

Diagrama de Alocação da CPU & Recurso Compartilhado (0 a 15 u.t.)

Destaque: Inversão de Prioridade ocorre no intervalo [6, 10]



⚡ Rastreamento Procedimental por Marcos de Eventos (Sem desenhar à mão)

Intervalo	CPU	Estado do Escalonador e Transições Críticas de Estado	Estado da PMC
[0, 2]	t1 (p=2)	Apenas t1 está no sistema. Roda 2 u.t. de CPU (restam 3 u.t.). No instante **t = 2**, solicita e adquire a PMC.	**Adquirida por t1**
[2, 3]	t1 (p=2)	t1 executa 1 u.t. dentro da PMC (ainda precisa de mais 1 u.t. de CPU dentro da seção crítica para liberar).	**Ocupada por t1**
t = 3	—	**Ingresso de t2 (p=3)**: Como p(t2)=3 > p(t1)=2, **t2 preempta t1**! t1 volta a prontas mantendo o lock da PMC.	**Ocupada por t1**
[3, 5]	t2 (p=3)	t2 executa 2 u.t. de CPU (restam 4 de suas 6 u.t. de duração).	**Ocupada por t1**
t = 5	—	**Ingresso de t3 (p=5)**: Como p(t3)=5 > p(t2)=3, **t3 preempta t2**! t3 assume a CPU. Este é o **início de t3**.	**Ocupada por t1**
[5, 6]	t3 (p=5)	t3 executa 1 u.t. de CPU (de 5 até 6). Em **t = 6** (1 u.t. após início), t3 solicita a PMC.	**Ocupada por t1**
t = 6	—	t3 requisita a PMC, mas a PMC está com t1. **t3 é BLOQUEADA** aguardando o recurso!	**t3 entra na fila**
[6, 10]	t2 (p=3)	**A Inversão de Prioridade**: Sem herança de prioridades, prontas são {t2(p=3), t1(p=2)}. O escalonador escolhe t2. t2 roda suas 4 u.t. restantes e **termina sua execução em t = 10**.	**Ocupada por t1**
[10, 11]	t1 (p=2)	Resta t1 pronta. t1 volta à CPU e roda a 1 u.t. de CPU que faltava dentro da PMC. Em **t = 11, t1 libera a PMC**!	**Liberada em t=11**
t = 11	—	A liberação acorda t3: **t3 recebe a PMC no instante 11**. Por ter p=5 > p(t1)=2, t3 preempta t1 imediatamente.	**Passada para t3**
[11, 13]	t3 (p=5)	t3 executa suas 2 u.t. de CPU na PMC. Em **t = 13, t3 libera a PMC** e encerra sua execução total (1 + 2 = 3 u.t.).	**Liberada em t=13**
[13, 15]	t1 (p=2)	t1 executa suas 2 u.t. finais fora da seção crítica e termina em t = 15.	Livre

Por que ocorreu a Inversão de Prioridade? Porque a tarefa de prioridade média (**t2**) executou durante 4 unidades de tempo enquanto a tarefa de prioridade mais alta do sistema (**t3**) ficou bloqueada no limbo esperando pela de menor prioridade (**t1**). Caso existisse o mecanismo de *Herança de Prioridades (Priority Inheritance)*, t1 teria herdado a prioridade 5 de t3 ao ser requisitada em t=6, rodando de imediato em [6, 7], liberando a PMC em t=7 e permitindo que t3 concluísse em t=9!